

Our research in business goals has led us to adopt the categories shown in the list that follows. These categories can be used as an aid to brainstorming and elicitation. By employing the list of categories, and asking the stakeholders about possible business goals in each category, some assurance of coverage is gained.

1. Growth and continuity of the organization
2. Meeting financial objectives
3. Meeting personal objectives
4. Meeting responsibility to the employees
5. Meeting responsibility to society
6. Meeting responsibility to the state
7. Meeting responsibility to the shareholders
8. Managing market position
9. Improving business processes
10. Managing the quality and reputation of products
11. Managing change in the environment over time

19.4 Capturing ASRs in a Utility Tree

In a perfect world, the techniques described in Sections 19.2 and 19.3 would be applied early on in your development process: You would interview the key stakeholders, elicit their business goals and driving architectural requirements, and have them prioritize all of these inputs for you. Of course, the real world, lamentably, is less than perfect. It is often the case that you do not have access to these stakeholders when you need them, for organizational or business reasons. So what do you do?

Architects can use a construct called a *utility tree* when the “primary sources” of requirements are not available. A utility tree is a top-down representation of what you, as an architect, believe to be the QA-related ASRs that are critical to the success of the system.

A utility tree begins with the word “Utility” as the root node. Utility is an expression of the overall “goodness” of the system. You then elaborate on this root node by listing the major QAs that the system is required to exhibit. (You might recall that we said in Chapter 3 that QA names by themselves were not very useful. Never fear—they are only being used as intermediate placeholders for subsequent elaboration and refinement!)

Under each QA, record specific refinements of that QA. For example, performance might be decomposed into “data latency” and “transaction throughput” or, alternatively, “user wait time” and “time to refresh web page.” The refinements that you choose should be the ones that are relevant to your system. Under each refinement, you can then record the specific ASRs, expressed as QA scenarios.

Once the ASRs are recorded as scenarios and placed at the leaves of the tree, you can evaluate these scenarios against two criteria: the business value of the candidate scenario and the technical risk of achieving it. You can use any scale you like, but we find that a simple “H” (high), “M” (medium), and “L” (low) scoring system suffices for each criterion. For business

value, “high” designates a must-have requirement, “medium” identifies a requirement that is important but would not lead to project failure were it omitted, and “low” describes a nice requirement to meet but not something worth much effort. For technical risk, “high” means that meeting this ASR is keeping you awake at night, “medium” means meeting this ASR is concerning but does not carry a high risk, and “low” means that you have confidence in your ability to meet this ASR.

Table 19.1 shows a portion of an example utility tree. Each ASR is labeled with an indicator of its business value and its technical risk.

TABLE 19.1 Tabular Form of the Utility Tree for a System in the Healthcare Space

Quality Attribute	Attribute Refinement	ASR Scenario
Performance	Transaction response time	A user updates a patient’s account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 seconds. (H, H)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M, M)
Usability	Proficiency training	A new hire with two or more years’ experience in the business can learn, with 1 week of training, to execute any of the system’s core functions in less than 5 seconds. (M, L)
	Efficiency of operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process with no input errors. (M, M)
Configurability	Data configurability	A hospital increases the fee for a particular service. The configuration team makes and tests the change in 1 working day; no source code needs to change. (H, L)
Maintainability	Routine changes	A maintainer encounters response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H, M)
		A reporting requirement requires a change to the report-generating metadata. Change is made and tested in 4 person-hours of effort (M, L)
	Upgrades to commercial components	The database vendor releases a new major version that is successfully tested and installed in less than 3 person-weeks. (H, M)
Security	Adding new feature	A feature that tracks blood bank donors is created and successfully integrated within 2 person-months. (M, M)
	Confidentiality	A physical therapist is allowed to see that part of a patient’s record dealing with orthopedic treatment, but not other parts or any financial information. (H, M)
	Resisting attacks	The system repels an unauthorized intrusion attempt and reports the attempt to authorities within 90 seconds. (H, M)
Availability	No down time	The database vendor releases new software, which is hot-swapped into place, with no downtime. (H, L)
		The system supports 24/7/365 web-based account access by patients. (M, M)

Once you have a utility tree filled out, you can use it to make important checks. For instance:

- A QA or QA refinement without any ASR scenario is not necessarily an error or omission that needs to be rectified, but rather an indication you should investigate whether there are unrecorded ASR scenarios in that area.
- ASR scenarios that receive a (H, H) rating are obviously the ones that deserve the most attention from you; these are the most significant of the significant requirements. A very large number of these scenarios might be a cause for concern regarding whether the system is, in fact, achievable.

19.5 Change Happens

Edward Berard said, “Walking on water and developing software from a specification are both easy if both are frozen.” Nothing in this chapter should be taken to assume that such a miraculous state of affairs is likely to exist. Requirements—whether captured or not—change all the time. Architects have to adapt and keep up, to ensure that their architectures are still the right ones that will bring success to the project. In Chapter 25, where we discuss architecture competence, we’ll advise that architects need to be great communicators, and this means great bidirectional communicators, taking in as well as supplying information. Always keep a channel open to the key stakeholders who determine the ASRs so you can keep up with changing requirements. The methods offered in this chapter can be applied repetitively to accommodate change.

Even better than keeping up with change is staying one step ahead of it. If you get wind of a change to the ASRs, you can take preliminary steps to design for it, as an exercise to understand the implications. If the change will be prohibitively expensive, sharing that information with the stakeholders will be a valuable contribution, and the earlier they know it, the better. Even more valuable might be suggestions about changes that would do (almost) as well in meeting the goals but without breaking the budget.

19.6 Summary

Architectures are driven by architecturally significant requirements. An ASR must have:

- A profound impact on the architecture. Including this requirement will likely result in a different architecture than if it were not included.
- A high business or mission value. If the architecture is going to satisfy this requirement—potentially at the expense of not satisfying others—it must be of high value to important stakeholders.